# React Apps

# React

- **What is React?**
  - **A JavaScript library for building user interfaces**
  - **Originally developed by Facebook in 2011, open-sourced in 2013**
  - **Designed for building fast, interactive web apps**
- **Why do people like it?**
  - **Looks really cool**
  - **Component-based architecture → Reusable, modular code**
  - **Virtual DOM → Fast updates & rendering**
- **Where is React used?**
  - **Facebook, Instagram, Netflix, Airbnb, Uber, and thousands of startups**

# Node.js

- Node.js is a runtime environment that lets you run JavaScript on your computer, not just in the browser
- Lets developers use JavaScript for backend servers, command-line tools, and more
- React is built with modern JavaScript tools that require Node.js to work
- Download here: https://nodejs.org/en/download
  - Scroll down and choose “prebuilt Node.js”

# Vite

- Vite is a modern frontend build tool and development server
- We use Vite to set up and run the React app
- Vite = fast in French
- React apps run in Vite start really quickly

# Vite React App Structure

```
my-vite-app/
├── public/                 ← Static assets (icons, robots.txt)
│   └── vite.svg
├── src/
│   ├── assets/             ← Images/styles to be processed by Vite
│   ├── components/         ← Your custom React components
│   │   └── MyComponent.tsx
│   ├── App.css             ← Styles for App component
│   ├── App.tsx             ← Root React component
│   ├── main.tsx            ← React entry point (replaces index.js)
│   └── vite-env.d.ts       ← TypeScript definitions for Vite
├── index.html              ← The entry point for the browser (now in root!)
├── package.json            ← Project metadata and dependencies
├── tsconfig.json           ← TypeScript configuration
└── vite.config.ts          ← Vite-specific settings/plugins
```

# Creating a Vite React App

1. Make a folder (i.e. my-app)

2. Open the folder in Cursor (so your terminal is in the folder)

3. Scaffold the app - In the terminal run
   npm create vite@latest . -- --template react-ts
   - The dot means make the app inside this folder you are in
   - --template react-ts means the language is TypeScript

4. Install – in the terminal run npm install
   - Downloads all React and Vite packages

5. Run the app - In the terminal, run npm run dev

# React Components

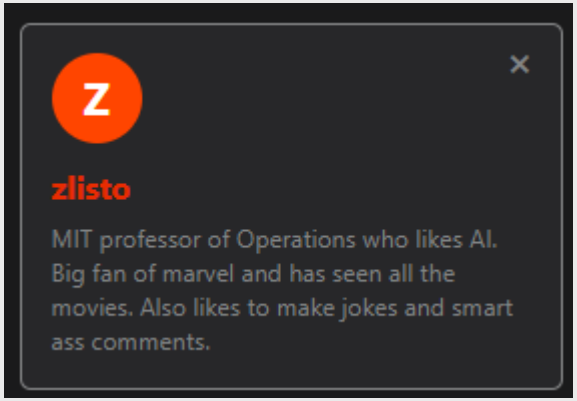
- A component is a reusable piece of user interface
- Components are JavaScript or TypeScript files (.js, .tsx)
- React apps are built by combining components like Lego blocks
- Example: this component will produce some nice HTML formatted text that says This is my custom component!

```
function MyComponent() {
  return <p>This is my custom component!</p>;
}

export default MyComponent;
```

# Example: AgentCard

```
const AgentCard = ({ agent, onDelete }) => {
  return (
    <div className="agent-card">
      <div className="agent-avatar" style={{ backgroundColor: agent.color }}>
        {agent.name.charAt(0).toUpperCase()}
      </div>
      <div className="agent-info">
        <h3 style={{ color: agent.color }}>{agent.name}</h3>
        <p>{agent.bio}</p>
      </div>
      <button
        className="delete-agent"
        onClick={() => onDelete(agent.id)}
      >
        ×
      </button>
    </div>
  );
};
```



# Example: App

- App is the main component that renders everything inside your app (this is in the App.tsx file)
- This component uses other components

```
export default function App() {
  const fileInputRef = useRef<HTMLInputElement>(null)
  const [lectureDetails, setLectureDetails] = useState('')
  const [speakingStyle, setSpeakingStyle] = useState('')
  const [slides, setSlides] = useState<Slide[]>([])
  const [pdfName, setPdfName] = useState<string | null>(null)

  const [busy, setBusy] = useState<Busy>('idle')
  const [progress, setProgress] = useState(0)
  const [progressNote, setProgressNote] = useState('')
  const [error, setError] = useState<string | null>(null)

  const [dragOver, setDragOver] = useState(false)

  const geminiConfigured = Boolean(getGeminiApiKey())
  const elevenConfigured = Boolean(getElevenLabsApiKey())

  const [ttsProvider, setTtsProvider] = useState<'gemini' | 'elevenlabs'>('gemini')
  const [geminiVoice, setGeminiVoice] = useState('Kore')
  const [elevenVoices, setElevenVoices] = useState<ElevenVoice[]>([])
  const [elevenVoiceId, setElevenVoiceId] = useState(() => getDefaultElevenLabsVoiceId())

  useEffect(() => {
    if (!elevenConfigured || ttsProvider !== 'elevenlabs') return
    let cancelled = false
    void (async () => {
      try {
        const v = await fetchElevenLabsVoices(getElevenLabsApiKey())
        if (cancelled) return
        setElevenVoices(v)
        if (!getDefaultElevenLabsVoiceId() && v[0]) setElevenVoiceId(v[0].voice_id)
```

# main.tsx

- In the main.tsx file, your App component is rendered to run your app

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.tsx'

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

# Coding Session

- We will build a simple React app
- We will try to make the style look cool
- Give the app some simple AI functionality